

Documentación Técnica — Framework E2E con Playwright + TypeScript

Documentación de referencia del diseño, las decisiones técnicas y el funcionamiento del proyecto, incluyendo las alternativas evaluadas con sus ventajas y desventajas.

Índice

1. Alcance
2. Qué es el testing E2E y dónde encaja
3. Por qué Playwright (vs Selenium vs Cypress)
4. Por qué TypeScript (vs JavaScript)
5. Anatomía del proyecto: cada carpeta y su porqué
6. Page Object Model a fondo
7. BasePage: cuándo la herencia sí conviene
8. Componentes reutilizables: composición sobre herencia
9. Fixtures de Playwright
10. Datos de prueba: centralización y patrón Builder
11. Configuración por ambiente
12. Estrategia de selectores (locators)
13. Esperas: auto-wait y aserciones web-first
14. Data-Driven Testing
15. Verificación de negocio: el checkout de totales
16. Aislamiento y paralelismo (la raíz del flakiness)
17. La configuración de Playwright, línea por línea
18. Reporting y trace viewer
19. Cross-browser
20. CI/CD: el workflow explicado
21. Extensiones sugeridas
22. Glosario
23. Próximos pasos

1. Alcance

Este proyecto es un **framework de automatización E2E**: un conjunto de código y convenciones que nos permite escribir, organizar y ejecutar pruebas automatizadas que manejan un navegador como lo haría un usuario real.

La app que probamos es **SauceDemo** (saucedemo.com), una tienda de demostración. La elegimos porque:

- Es **pública y gratuita**, no necesitás credenciales especiales.
- Es **estable**: no cambia todo el tiempo, así que los tests no se rompen por causas ajenas.
- Tiene un **flujo de negocio completo**: login, catálogo, ordenamiento, carrito y checkout con cálculo de totales.
- Expone atributos **data-test** en el HTML, que son la mejor práctica para localizar elementos (lo vemos en la sección 12).

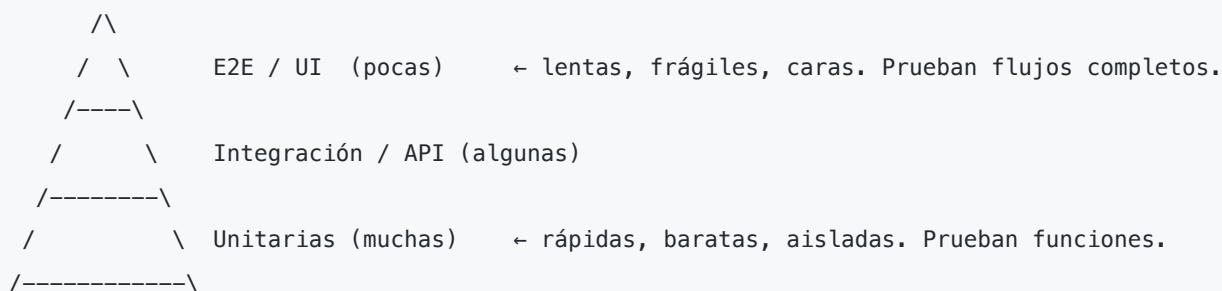
Nota de honestidad técnica: SauceDemo NO tiene un backend/API real (su estado vive en el navegador). Por eso una técnica avanzada como "preparar datos por API" no aplica acá, y la vamos a ver en el **Proyecto 2** (testing de API). Saber reconocer las limitaciones de tu app bajo prueba es parte del criterio de un QA senior.

2. Qué es el testing E2E y dónde encaja

End-to-end (E2E) significa probar un flujo completo de la aplicación, de punta a punta, como lo haría un usuario: abrir el navegador, loguearse, agregar productos, pagar. Verifica que **todas las piezas integradas funcionen juntas** (frontend + backend + base de datos).

La pirámide de automatización

No todo se prueba con E2E. La **pirámide de testing** describe cómo distribuir las pruebas:



- **Base (unitarias):** prueban una función o clase aislada. Corren en milisegundos. Cuando fallan, te dicen exactamente qué se rompió.

- **Medio (API/integración):** prueban que los servicios se comuniquen bien. Más rápidas y estables que la UI.
- **Cima (E2E/UI):** prueban el flujo completo desde la interfaz. Son **valiosas pero costosas**: lentas y sensibles a cualquier cambio visual.

Este proyecto vive en la cima de la pirámide. La regla clave: automatizá en E2E solo lo que **realmente necesitás ver funcionar de punta a punta** (los flujos críticos de negocio), y empujá el resto hacia capas más bajas. Un error común de principiante es querer probar TODO por la UI, lo que produce suites lentas e inestables.

3. Por qué Playwright (vs Selenium vs Cypress)

Elegimos **Playwright**. Estas son las tres opciones principales del mercado y por qué:

Criterio			
Creado por	Microsoft (2020)	Cypress.io (2017)	Comunidad (2004)
Arquitectura	Controla el navegador por protocolo (fuera del browser)	Corre dentro del navegador	WebDriver (estándar W3C)
Auto-wait (esperas automáticas)	✅ Sí, muy robusto	✅ Sí	❌ No (hay que programarlas)
Multi-navegador	Chromium, Firefox, WebKit	Chrome, Firefox, WebKit (más limitado)	Todos
Multi-lenguaje	JS/TS, Python, Java, .NET	Solo JavaScript/TypeScript	Muchísimos
Multi-tab / multi-dominio	✅ Nativo	❌ Limitado (por su arquitectura)	✅
Testing de API integrado	✅ Sí	Parcial	❌
Trace viewer / debugging	✅ Excelente (time-travel)	✅ Muy bueno	❌ Básico
Velocidad	Muy rápida	Rápida	Más lenta
Curva de setup	Baja (todo incluido)	Baja	Alta (armás todo a mano)

Cuándo no conviene cada uno

- **Cypress NO**, si necesito flujos que salten entre dominios (ej: login con Google/SSO), múltiples pestañas, o un lenguaje que no sea JS. Su arquitectura de "correr dentro del navegador" lo limita.
- **Selenium NO**, para un proyecto nuevo sin requisito de compatibilidad legacy: me obliga a construir a mano (esperas, paralelismo, reportes) lo que los otros traen resuelto.

- **Playwright NO**, si el equipo ya tiene una suite madura y productiva en otra herramienta y el costo de migrar no se justifica.

Por qué Playwright acá

1. **Auto-wait robusto** → menos flakiness "gratis" (sección 13).
 2. **Trace viewer** → diagnóstico de fallos sin re-ejecutar (sección 18).
 3. **Web + API en la misma herramienta** → nos servirá para el Proyecto 2.
 4. **Cross-browser real** con los tres motores (Chromium, Firefox, WebKit).
 5. Es, hoy, el estándar de facto para proyectos nuevos y lo más demandado en búsquedas.
-

4. Por qué TypeScript (vs JavaScript)

Playwright funciona con JavaScript o TypeScript. Elegimos **TypeScript**.

TypeScript = JavaScript + tipos estáticos. Antes de correr el código, un compilador verifica que los tipos sean coherentes.

Pros de TypeScript en un framework de testing

- **Errores en tiempo de escritura, no en producción.** Si un método espera un `User` y le pasás un string, el editor te avisa al instante en rojo, antes de correr nada.
- **Autocompletado real.** Al escribir `loginPage.` el editor te sugiere `login()`, `loginWith()`, etc. Esto acelera muchísimo y reduce errores de tipeo.
- **Refactors seguros.** Si renombrás un método, TypeScript te marca todos los lugares que hay que actualizar.
- **Documentación viva.** La firma `login(user: User): Promise<void>` ya te dice qué recibe y qué devuelve.

Contras

- **Curva de aprendizaje** si venís solo de JS puro.
- Un **paso de compilación**, aunque Playwright lo maneja de forma transparente.

En este proyecto activamos `"strict": true` en `tsconfig.json`, el modo más exigente: nada de tipos implícitos `any`, chequeo de null, etc. Y tenemos `npm run typecheck` que valida todo sin correr los tests.

5. Anatomía del proyecto: cada carpeta y su porqué

El principio rector es **Separación de Responsabilidades** (cada cosa en su lugar, cada archivo con un único motivo para cambiar).

```
src/  
├─ config/      → CÓMO se conecta el framework al ambiente (URLs, flags)  
├─ data/        → QUÉ datos usamos (usuarios, datos de formularios)  
├─ components/  → PIEZAS de UI reutilizables (el header)  
├─ pages/       → las PANTALLAS de la app (Page Objects)  
└─ fixtures/    → el PEGAMENTO que inyecta los Page Objects a los tests  
  
tests/         → los tests en sí (el QUÉ verificamos), agrupados por feature  
docs/          → esta documentación  
.github/       → automatización de CI
```

¿Por qué separar `src/` (código de soporte) de `tests/` (los tests)?

Porque tienen roles distintos. En `tests/` vive **qué** se verifica (la intención de negocio). En `src/` vive **cómo** se interactúa con la app (los detalles técnicos). Esta separación es la que permite que, cuando la UI cambia, toques `src/` y los `tests/` queden intactos.

¿Por qué agrupar los tests por feature (`auth` , `cart` , `checkout`)?

Porque escala. Cuando tengas 200 tests, agruparlos por funcionalidad hace trivial encontrar, correr o mantener un área específica. Además permite asignar **ownership** (qué equipo es dueño de qué carpeta), algo clave en organizaciones grandes.

6. Page Object Model a fondo

El **Page Object Model (POM)** es el patrón central de este framework. La idea: **cada página de la app se representa como una clase** que encapsula:

1. Sus **localizadores** (dónde están los elementos).
2. Sus **acciones de negocio** (qué se puede hacer ahí).

El antipatrón (lo que evitamos)

Sin POM, un test se ve así — con los selectores incrustados:

```
// ❌ MAL: selectores y lógica mezclados en el test
test('login', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.locator('[data-test="username"]').fill('standard_user');
  await page.locator('[data-test="password"]').fill('secret_sauce');
  await page.locator('[data-test="login-button"]').click();
  await expect(page.locator('[data-test="title"]').toHaveText('Products');
});
```

Problemas:

- Si el selector del usuario cambia, hay que corregirlo en **cada test** que hace login (pueden ser 50).
- El test es difícil de leer: no se entiende la intención entre tanto detalle técnico.

Con POM (lo que hacemos)

```
// ✅ BIEN: el test habla de negocio, no de selectores
test('login', async ({ loginPage, inventoryPage }) => {
  await loginPage.goto();
  await loginPage.login(USERS.standard);
  await expect(inventoryPage.titleLocator()).toHaveText('Products');
});
```

Y el `LoginPage` encapsula los detalles:

```
export class LoginPage extends BasePage {
  private readonly usernameInput = this.page.locator('[data-test="username"]');
  // ...
  async login(user: User): Promise<void> {
    await this.usernameInput.fill(user.username);
    await this.passwordInput.fill(user.password);
    await this.loginButton.click();
  }
}
```

Beneficios:

- **Un solo lugar para mantener.** Si cambia el selector, se corrige en `LoginPage` y listo.
- **Tests legibles.** `loginPage.login(USERS.standard)` se entiende sin ser técnico.
- **Reutilización.** El mismo `login()` sirve para decenas de tests.

Reglas de oro del POM (cómo lo hacemos "bien" y no solo "a medias")

1. **Los page objects NO contienen aserciones.** Exponen datos o `Locator` s, y el **test** hace las aserciones. Así separamos "interactuar con la página" de "verificar expectativas". (Ver cómo exponemos `errorLocator()` en vez de `assert` adentro.)
2. **Los métodos hablan de negocio**, no de mecánica: `login()` , no `typeUsernameAndPassword()` .
3. **Los tests nunca conocen un selector.** Si ves un `[data-test=...]` en un archivo de `tests/` , algo está mal.

7. BasePage: cuándo la herencia sí conviene

Todas las páginas heredan de `BasePage` :

```
export abstract class BasePage {
  protected abstract readonly path: string; // cada página define su ruta
  constructor(protected readonly page: Page) {}
  async goto(): Promise<void> {
    await this.page.goto(this.path);
  }
}
```

¿Por qué herencia acá? Porque hay comportamiento **genuinamente común** a todas las páginas: guardar la referencia a `page` y navegar por una ruta relativa. Es una relación "es un": `LoginPage` **es una** página.

`abstract` significa que `BasePage` no se puede instanciar sola (no existe "una página genérica"): solo sirve para ser heredada. Y `path` es `abstract` , así que **obliga** a cada página hija a declarar su ruta. Es el compilador ayudándonos a no olvidarnos.

Regla práctica: usá herencia para lo que es "es un" y común a todos (páginas ↔ `BasePage`). Usá **composición** para lo que es "tiene un" (una página **tiene un** header). Lo vemos ahora.

8. Componentes reutilizables: composición sobre herencia

El header (ícono del carrito, badge de cantidad, menú) aparece en el inventario, el carrito y el checkout. **No es una página:** es una porción de UI que se repite.

Modelarlo como **componente** y reutilizarlo por **composición** evita duplicar sus localizadores en cada página:

```
export class InventoryPage extends BasePage {
  readonly header: HeaderComponent; // ← "tiene un" header (composición)
  constructor(page: Page) {
    super(page);
    this.header = new HeaderComponent(page);
  }
}
```

Desde el test:

```
await inventoryPage.header.getCartCount(); // reutiliza el componente
await inventoryPage.header.openCart();
```

¿Por qué composición y no herencia acá? Porque el inventario **no es un** header; **tiene un** header. Si intentáramos resolver esto con herencia, terminaríamos con jerarquías absurdas. La composición es más flexible: el mismo `HeaderComponent` lo usan tres páginas distintas sin acoplarlas entre sí.

Este es un principio de diseño clásico: "**preferí composición por sobre herencia**". La herencia acopla fuerte; la composición arma piezas independientes que se combinan.

9. Fixtures de Playwright

Los **fixtures** son el mecanismo idiomático de Playwright para **preparar lo que un test necesita** e inyectárselo.

El problema que resuelven

Sin fixtures, cada test tendría que instanciar sus page objects a mano:

```
// ❌ Repetitivo: este boilerplate se repite en cada test
test('...', async ({ page }) => {
  const loginPage = new LoginPage(page);
  const inventoryPage = new InventoryPage(page);
  // ...recién ahora empieza la lógica del test
});
```

La solución

Extendemos el `test` de Playwright para que **inyecte los page objects ya construidos**:


```
export const test = base.extend<Pages>({
  loginPage: async ({ page }, use) => { await use(new LoginPage(page)); },
  inventoryPage: async ({ page }, use) => { await use(new InventoryPage(page)); },
  // ...
});
```

Y el test los recibe listos por desestructuración:

```
test('...', async ({ loginPage, inventoryPage }) => {
  // directo a la lógica, sin boilerplate
});
```

Ventajas sobre las alternativas

Alternativa	Problema
Instanciar en cada test	Repetición de boilerplate
<code>beforeEach</code> gigante	Se ejecuta todo aunque el test use la mitad; difícil de componer
Clase base de test	Rígida; herencia forzada; no compone bien
Fixtures	Lazy (solo construye lo que el test pide), compone, es lo idiomático

Fixtures "lazy": si un test pide solo `{ loginPage }`, Playwright construye **solo** `LoginPage`. No malgasta creando lo que no se usa.

El fixture `LoggedInInventory`

Creamos un fixture de conveniencia que **entrega el inventario ya con el usuario logueado**:

```
loggedInInventory: async ({ page }, use) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login(USERS.standard);
  await use(new InventoryPage(page));
},
```

Así, los tests que **no** prueban el login (ordenamiento, carrito, checkout) arrancan directamente autenticados, sin repetir los pasos de login. Encapsula la precondition.

Optimización avanzada (para más adelante): para suites grandes, se puede logear **una sola vez** y reutilizar la sesión guardada (`storageState`), evitando repetir el login por UI en cada test. Lo dejamos anotado como mejora; en este proyecto priorizamos claridad.

10. Datos de prueba: centralización y patrón Builder

Centralización (`users.ts`)

Todos los usuarios viven en un solo archivo:

```
export const USERS = {
  standard: { username: 'standard_user', password: PASSWORD },
  lockedOut: { username: 'locked_out_user', password: PASSWORD },
  // ...
};
```

¿Por qué? Si un dato cambia, se corrige en un solo lugar. Y los tests se leen mejor: `USERS.lockedOut` dice más que `'locked_out_user'`.

Patrón Builder (`checkout.data.ts`)

Para los datos de checkout usamos el patrón **Builder**:

```
new CheckoutInfoBuilder().withPostalCode('').build()
```

El problema que resuelve: a veces un test necesita "los datos válidos, PERO con el código postal vacío". Sin builder, tendrías que reescribir el objeto entero cada vez. Con builder, declararás **solo la variación** que te importa y todo lo demás toma un valor por defecto válido:

```
// "datos válidos por defecto"
new CheckoutInfoBuilder().build()

// "datos válidos pero sin código postal" (para probar la validación)
new CheckoutInfoBuilder().withPostalCode('').build()
```

Cada método (`withPostalCode`) devuelve `this` , lo que permite **encadenar** llamadas (interfaz fluida). Y `build()` devuelve una **copia** (`{ ...this.info }`) para que dos tests no compartan y modifiquen el mismo objeto (fuente clásica de bugs entre tests).

11. Configuración por ambiente

La URL base **no está hardcodeda**. Vive en `src/config/env.ts` y se lee de una variable de entorno:

```
export const ENV: EnvConfig = {
  baseUrl: process.env.BASE_URL ?? DEFAULT_BASE_URL,
};
```

¿Por qué? Porque la misma suite debe poder correr contra **dev, staging o producción** sin cambiar código:

```
BASE_URL=https://staging.mi-app.com npm test
```

En `playwright.config.ts`, esa `baseUrl` se usa para que los page objects naveguen con **rutas relativas** (`/inventory.html`) en vez de URLs absolutas. Así, cambiar de ambiente es cambiar **una** variable.

Secretos: el archivo `.env` está en `.gitignore` — nunca se sube al repo. En CI, los secretos viven en el gestor de secretos de la plataforma (GitHub Secrets). Hardcodear una credencial en el código es un error de seguridad clásico.

12. Estrategia de selectores (locators)

Un **selector** (o localizador) es cómo le decimos a Playwright "este elemento". Elegir bien el selector es **la decisión que más impacta en la estabilidad** de la suite.

Jerarquía de preferencia (de mejor a peor)

1. **Atributos de test dedicados** (`data-test`, `data-testid`) — **los mejores**. Son marcas que el equipo pone a propósito para testing. No cambian con el estilo ni el layout.

```
page.locator('[data-test="login-button"]')
```

2. **Selectores por rol y texto accesible** (`getByRole`, `getByLabel`) — muy buenos. Además validan accesibilidad.

```
page.getByRole('button', { name: 'Add to cart' })
```

3. **Selectores CSS** por clases semánticas — aceptables.

```
page.locator('.inventory_item')
```

4. **XPath largos atados a la estructura** — **los peores**. Frágiles: se rompen ante cualquier cambio del DOM.

```
// ❌ Evitar: se rompe si alguien agrega un <div> en el medio
page.locator('/html/body/div[2]/div[3]/form/button')
```

Cómo lo aplicamos

SauceDemo tiene excelentes `data-test`, así que los usamos como primera opción:

```
this.loginButton = page.locator('[data-test="login-button"]');
```

Cuando necesitamos localizar algo **dinámico** (un producto por su nombre), combinamos: filtramos el contenedor por texto y usamos el rol del botón interno:

```
const item = this.items.filter({ hasText: productName });
await item.getByRole('button', { name: 'Add to cart' }).click();
```

Esto es robusto: no depende de la posición del producto en la lista ni de un `data-test` que incluya el slug del producto.

Tip para tu trabajo real: si la app no tiene `data-test`, **pedile a los desarrolladores que los agreguen** a los elementos clave. Es un cambio trivial para ellos y multiplica la estabilidad de tu automatización. Que un QA proponga esto es una señal de seniority.

13. Esperas: auto-wait y aserciones web-first

Esta es **la causa número uno de tests flaky** (inestables), así que prestale atención.

El problema de las esperas fijas

En una app moderna, los elementos aparecen de forma **asincrónica** (tras una llamada de red, un render, una animación). Si tu test intenta hacer click antes de que el elemento esté listo, falla.

La tentación del principiante es agregar una espera fija:

```
// ❌ NUNCA hagas esto
await page.waitForTimeout(3000); // "esperá 3 segundos"
await boton.click();
```

Por qué está mal: una espera fija siempre falla por uno de dos lados:

- Si el elemento aparece en 0.5s, **desperdiciás 2.5s** en cada test → la suite se vuelve lentísima.
- Si algún día tarda 3.5s (red lenta), el test **falla** aunque la app funcione → **falso negativo, test flaky**.

La solución de Playwright: auto-wait

Playwright, **antes de cada acción**, espera automáticamente a que el elemento esté listo (visible, habilitado, estable). No tenés que programar la espera:

```
// ✅ Playwright espera solo, lo necesario, ni más ni menos
await boton.click();
```

Aserciones web-first

Lo mismo para las verificaciones. Las aserciones `expect(locator)...` **reintentan** hasta que se cumplen o se vence el timeout:

```
// ✅ Espera/reintenta hasta que el título diga "Products"
await expect(inventoryPage.titleLocator()).toHaveText('Products');
```

Compará con el antipatrón, que lee el valor **una sola vez** (y falla si el DOM todavía no se actualizó):

```
// ❌ Lee una vez, sin reintentar
const texto = await titulo.textContent();
expect(texto).toBe('Products');
```

Por eso en los page objects exponemos **Locator** s (no strings) para las aserciones: así el test puede usar `expect(locator)` y aprovechar el auto-wait.

Regla de oro: nunca esperes por tiempo, esperá por condición. En todo este proyecto **no hay un solo `waitForTimeout`**.

¿Y si de verdad necesito esperar algo específico?

Esperá el evento real, no un tiempo. Ejemplo: esperar que una llamada de red responda antes de seguir:

```
await page.waitForResponse(resp => resp.url().includes('/api/items') && resp.status() === 200);
```

14. Data-Driven Testing

En `login.spec.ts` probamos varios casos negativos de login. En lugar de escribir un test por cada uno (copy-paste), definimos los datos en un array y **generamos** un test por caso:

```
const casosNegativos = [
  { caso: 'usuario bloqueado', username: '...', password: '...', errorEsperado: '...' },
  { caso: 'password incorrecta', username: '...', password: '...', errorEsperado: '...' },
  // ...
];

for (const tc of casosNegativos) {
  test(`login fallido: ${tc.caso}`, async ({ loginPage }) => {
    await loginPage.loginWith(tc.username, tc.password);
    await expect(loginPage.errorLocator()).toContainText(tc.errorEsperado);
  });
}
```

Ventajas:

- **Sin duplicación:** la lógica del test está una sola vez.
- **Reporte claro:** cada caso aparece como un test independiente ("login fallido: usuario bloqueado"), así sabés exactamente cuál falló.
- **Extensible:** agregar un caso nuevo es agregar una línea al array.

Este patrón es el equivalente en código de las técnicas de diseño de casos que se usan en QA manual (partición de equivalencia, valores límite): una tabla de entradas y salidas esperadas.

15. Verificación de negocio: el checkout de totales

El test estrella (`checkout.spec.ts`) no solo "hace clicks y confirma que aparece un mensaje". Hace una **verificación de negocio real**: que la matemática de la compra sea correcta.

```
const itemTotal = await checkoutStepTwo.getItemTotal();
const tax = await checkoutStepTwo.getTax();
const total = await checkoutStepTwo.getTotal();

expect(Number((itemTotal + tax).toFixed(2))).toBe(total);
```

Por qué esto importa: un test que solo verifica "aparece la palabra Total" pasaría aunque el sitio cobre de más. Verificar que `Total = Item total + Impuesto` protege al usuario y al negocio de un bug real y costoso.

Detalle técnico — `toFixed(2)`: en JavaScript (y casi todos los lenguajes), la aritmética de punto flotante tiene imprecisiones: `0.1 + 0.2` da `0.30000000000000004`, no `0.3`. Por eso redondeamos a 2 decimales antes de comparar montos de dinero. Conocer esta trampa es un detalle que distingue a alguien que entiende cómo funciona el lenguaje.

16. Aislamiento y paralelismo (la raíz del flakiness)

Tests independientes

Cada test debe ser una **isla**: no depender de que otro corrió antes, no compartir estado. En este proyecto:

- Cada test arranca de cero (Playwright le da un contexto de navegador limpio, sin cookies ni sesión previa).
- El fixture `loggedInInventory` crea el estado autenticado **para ese test**, no lo hereda de otro.
- Los datos (`CheckoutInfoBuilder.build()`) se copian, no se comparten.

Paralelismo

En `playwright.config.ts` activamos `fullyParallel: true`. Playwright corre los tests en **paralelo**, repartidos entre varios *workers* (procesos). Por eso la suite completa (39 ejecuciones: 13 tests × 3 navegadores) termina en segundos.

La conexión clave: el paralelismo **solo funciona si los tests están aislados**. Si dos tests compartieran datos, al correr en paralelo se pisarían y fallarían de forma aleatoria (flaky). Es decir: **el aislamiento es lo que habilita la velocidad**.

Este es uno de los temas más "senior" que existen. Un QA Sr no solo escribe tests que pasan; escribe tests que pasan **de forma confiable, en paralelo, siempre**. El Proyecto 4 de tu roadmap está dedicado enteramente a esto.

17. La configuración de Playwright, línea por línea

`playwright.config.ts` es el cerebro del framework. Repasemos las decisiones:

```
testDir: './tests',           // dónde buscar los *.spec.ts
fullyParallel: true,           // corre tests en paralelo (ver sección 16)
forbidOnly: !!process.env.CI, // en CI, falla si quedó un test.only olvidado
retries: process.env.CI ? 2 : 0, // reintentos
workers: process.env.CI ? 4 : undefined, // procesos en paralelo
```

retries : 2 en CI, 0 en local. ¿Por qué la diferencia?

- En **CI**, 2 reintentos absorben inestabilidades puntuales de infraestructura (una red que hipó) y evitan romper un build por una causa ajena.
- En **local**, 0 reintentos **a propósito**: mientras desarrollás, querés VER el flakiness, no esconderlo. Un reintento que "arregla" un test flaky está tapando un problema real.

```

use: {
  baseUrl: ENV.baseUrl,           // URLs relativas en los page objects
  trace: 'on-first-retry',        // guarda la traza solo si reintenta
  screenshot: 'only-on-failure',  // captura solo si falla
  video: 'retain-on-failure',     // video solo si falla
  actionTimeout: 10_000,         // timeout por acción individual
}

```

trace: 'on-first-retry' es un balance inteligente: capturar la traza (que tiene un costo) **solo cuando algo falló y se reintenta**, no en cada corrida exitosa. Así tenés diagnóstico completo de los fallos sin penalizar la velocidad general.

screenshot / video: only/retain-on-failure : no llenamos el disco con evidencia de tests que pasaron; guardamos solo lo útil para diagnosticar los que fallaron.

```

projects: [
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  { name: 'webkit', use: { ...devices['Desktop Safari'] } },
]

```

Cada **project** es una configuración de ejecución. Acá, tres navegadores: el mismo test corre en los tres motores (sección 19).

18. Reporting y trace viewer

Configuramos dos reporters:

```

reporter: [
  ['html', { open: 'never' }], // reporte navegable
  ['list'],                   // salida en consola durante la corrida
]

```

- **list** : muestra el progreso en la terminal mientras corre (los ✓ verdes que viste).
- **html** : genera un reporte web navegable. Se abre con `npm run report`.

El trace viewer (la joya de Playwright)

Cuando un test falla y se reintenta, Playwright guarda un **trace**: una grabación completa de la ejecución que incluye, para **cada paso**:

- Una captura del DOM en ese instante (podés inspeccionarlo como si estuvieras ahí).
- Las llamadas de red que ocurrieron.

- Los logs de consola.
- Un timeline navegable ("time-travel debugging").

Por qué es tan valioso: sin esto, cuando un test falla en CI tenés que reproducirlo localmente para entender qué pasó (y a veces ni se reproduce). Con el trace, abrís la grabación de esa corrida específica y ves exactamente en qué paso y por qué falló. **Ahorra horas de debugging.** Este es un argumento fuerte a favor de Playwright.

19. Cross-browser

Los usuarios no usan todos el mismo navegador. Un bug puede aparecer solo en Safari (WebKit) y no en Chrome. Por eso corremos la misma suite en **tres motores de navegador**:

- **Chromium** → Chrome, Edge, Brave, Opera.
- **Firefox** → Firefox.
- **WebKit** → Safari (incluido iOS Safari).

Con la matriz de `projects`, escribís el test **una vez** y Playwright lo corre en los tres. Por eso 13 tests se convierten en 39 ejecuciones.

Balance de costo: cross-browser da confianza pero multiplica el tiempo. Estrategia práctica: en cada PR correr solo **Chromium** (rápido), y la matriz completa en la corrida nocturna o antes de release. Por eso tenemos el script `test:chromium` separado.

20. CI/CD: el workflow explicado

`.github/workflows/ci.yml` define un pipeline de **Integración Continua**. GitHub lo ejecuta automáticamente en cada `push` y cada Pull Request a `main`.

```
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
```

Los pasos, en un runner de Ubuntu limpio:

1. **Checkout** del código.
2. **Setup de Node** (con caché de npm para acelerar).
3. `npm ci` → instala dependencias de forma reproducible (usa el `package-lock.json` exacto, a diferencia de `npm install`).
4. `npm run typecheck` → verifica tipos antes de gastar tiempo corriendo tests.
5. **Instala Chromium** con sus dependencias de sistema.

6. **Corre la suite** en Chromium.

7. **Sube el reporte** como artifact descargable (incluso si los tests fallan, con `if: !cancelled()`, para poder diagnosticar).

Por qué esto es nivel Sr: automatizar sin CI es "media automatización". El valor de una suite se multiplica cuando corre **sola** en cada cambio, actuando como red de seguridad antes de que el código llegue a producción. Este workflow es la base del **Proyecto 3**, donde lo extenderemos (smoke bloqueante vs regresión nightly, matriz de navegadores, badges, etc.).

21. Extensiones sugeridas

Direcciones para extender el proyecto:

1. **Agregá un test:** que el botón "Add to cart" cambie a "Remove" después de agregar un producto.
2. **Nuevo Page Object:** modelá la página de detalle de un producto (`/inventory-item.html?id=4`) y probá que muestra el nombre y precio correctos.
3. **Rompé un selector a propósito** (cambiá un `data-test` por uno inexistente), corré el test, y **abrí el trace** para ver cómo se diagnostica el fallo.
4. **Data-driven:** convertí el test de ordenamiento en data-driven, con un array de las 4 opciones de orden.
5. **Nuevo usuario:** agregá un test con `problem_user` (que tiene bugs de UI) y observá qué falla y por qué.
6. **storageState:** investigá cómo loguearte una sola vez y reutilizar la sesión para acelerar la suite.

22. Glosario

- **E2E (end-to-end):** prueba de un flujo completo, de punta a punta, como un usuario real.
- **Page Object Model (POM):** patrón donde cada pantalla es una clase que encapsula sus localizadores y acciones.
- **Fixture:** mecanismo de Playwright para preparar e inyectar lo que un test necesita.
- **Locator:** referencia a un elemento del DOM. Es "perezoso": no busca el elemento hasta que se usa.
- **Auto-wait:** espera automática de Playwright antes de cada acción.
- **Aserción web-first:** `expect(locator)...` que reintenta hasta cumplirse o vencer el timeout.
- **Flaky (test flaky):** test que a veces pasa y a veces falla sin cambios reales. El enemigo número uno.
- **Selector / localizador:** la expresión que identifica un elemento (`data-test` , CSS, XPath).
- **data-test / data-testid:** atributo HTML puesto a propósito para testing; el localizador más estable.
- **Worker:** proceso que Playwright usa para correr tests en paralelo.
- **Trace:** grabación completa de una ejecución para diagnóstico (time-travel debugging).

- **CI/CD:** Integración/Entrega Continua; automatización que corre la suite en cada cambio.
 - **Builder:** patrón para construir objetos declarando solo lo que varía.
 - **Smoke test:** verificación rápida de que lo esencial funciona.
 - **Regresión:** re-ejecución de tests para asegurar que lo que funcionaba sigue funcionando.
-

23. Próximos pasos

Con esta base sólida, los siguientes construyen encima:

- **Proyecto 2 — Testing de API:** la capa que falta de la pirámide. Contratos, negativos, encadenamiento, validación de schema. Acá sí veremos "setup por API".
- **Proyecto 3 — CI/CD completo:** extender este workflow (smoke bloqueante, regresión nightly, matriz, badges, notificaciones).
- **Proyecto 4 — Estabilidad y flakiness:** el tema más senior. Crear flakiness a propósito, medirlo y eliminarlo.
- **Proyecto 5 — Visual regression + contract testing (Pact):** capas avanzadas.

Lo más importante: **este proyecto es tuyo y es real**. Podés explicar cada línea porque la construiste y la entendés. Eso es lo que te posiciona de verdad.